

Atividade Prática - Minha Lista de Filmes

Contexto

Durante a aula, vimos que dados armazenados apenas em uma `List` desaparecem quando o aplicativo é encerrado.

Agora você deverá criar um aplicativo para organizar **filmes que deseja assistir**, utilizando **SQLite para manter os dados salvos**.

O aplicativo deverá permitir cadastrar filmes e marcar aqueles que já foram assistidos.

Objetivo

Desenvolver um aplicativo Flutter utilizando:

```
None
MVVM
+
Provider
+
Service
+
SQLite
```

Os dados deverão continuar disponíveis mesmo depois que o aplicativo for fechado e aberto novamente.

Estrutura sugerida

Organize o projeto da seguinte forma:

```
None
lib/
```

```
|
|— main.dart
|
|— models/
|   └─ filme.dart
|
|— pages/
|   └─ pagina_inicial.dart
|
|— viewmodels/
|   └─ filme_viewmodel.dart
|
└─ services/
    └─ filme_service.dart
```

Parte 1 - Model

Crie uma classe chamada:

None
Filme

Ela deverá possuir:

None
id
titulo
assistido

Considere que:

None
id
→ será criado pelo SQLite

titulo

→ nome do filme

assistido

→ true ou false

Parte 2 - Banco de dados

Crie um banco chamado:

None

`filmes.db`

Dentro dele deverá existir a tabela:

None

`filmes`

Com a seguinte estrutura:

None

`id`

→ INTEGER

→ PRIMARY KEY

→ AUTOINCREMENT

`titulo`

→ TEXT

→ NOT NULL

`assistido`

→ INTEGER

→ NOT NULL

Lembre-se:

None

false → 0

true → 1

Parte 3 - Service

Crie:

None

filme_service.dart

O Service deverá ser responsável pelo acesso ao SQLite.

Implemente os métodos necessários para:

None

abrir/criar o banco

inserir um filme

listar os filmes

atualizar o status de um filme

Relacione com o CRUD:

None

INSERT

→ cadastrar filme

SELECT

→ buscar filmes

UPDATE

→ marcar/desmarcar como assistido

Parte 4 - ViewModel

Crie:

```
None  
filme_viewmodel.dart
```

A ViewModel deverá possuir uma:

```
None  
List<Filme>
```

Porém, atenção:

A List não deve ser responsável pela persistência.

Os dados deverão ser buscados através do FilmeService.

A ViewModel deverá permitir:

```
None  
carregar os filmes salvos  
  
adicionar um novo filme  
  
alterar o status de assistido
```

Sempre que necessário, atualize a interface utilizando:

```
None  
notifyListeners();
```

Lembre-se:

```
None  
notifyListeners()  
→ atualiza a interface  
  
SQLite  
→ mantém os dados salvos
```

Parte 5 - Interface

A página principal deverá possuir:

- um TextField para digitar o nome do filme;
- um ElevatedButton para cadastrar;
- uma lista mostrando os filmes;
- um Checkbox para marcar se o filme já foi assistido.

Exemplo:

```
None
Minha Lista de Filmes

[ Digite o nome do filme      ]

      [ Adicionar ]

-----

1  Interestelar      ☒
2  Homem-Aranha     ☐
3  Shrek             ☒
```

Quando assistido == true, o nome do filme também deverá aparecer **riscado**.

Parte 6 - Carregamento inicial

Quando o aplicativo abrir, os filmes já cadastrados deverão ser buscados automaticamente no SQLite.

Ou seja:

None

App abre

↓

ViewModel

↓

Service

↓

SQLite

↓

SELECT

↓

Filmes aparecem

Parte 7 - Teste obrigatório

Antes de finalizar, faça o seguinte teste:

1. Cadastre pelo menos **3 filmes**.
2. Marque pelo menos **1 filme como assistido**.
3. Feche/reinicie o aplicativo.
4. Abra novamente.

O resultado deverá continuar semelhante a:

None

☒ Interestelar

☐ Homem-Aranha

☐ Shrek

Se os filmes desaparecerem, a persistência ainda não está funcionando corretamente.

Regras da atividade

Utilize os conceitos trabalhados durante a aula.

O projeto deverá utilizar:

None

Model

View

ViewModel

Service

Provider

SQLite

Não faça o acesso ao SQLite diretamente na página.

O fluxo esperado é:

None

View

↓

ViewModel

↓

Service

↓

SQLite

E, para recuperar os dados:

None

SQLite

↓

Service

↓

ViewModel

↓

View